Grokking the Art of Recursive Problem-Solving

Introduction

Thinking recursively by breaking down complex problems into manageable parts is more than just a skill. It's a problem-solving art that is a "must-have" for every programmer. According to a 2022 survey by HackerRank, over `90%` of employers believe that a solid understanding of recursive algorithmic thinking is critical for candidates seeking roles in software development, data science, and artificial intelligence.

Several practical computational problems (e.g., traversing directories in a file system, parsing HTML documents, and performing efficient searches on data structures like trees and graphs) have recursive nature and require recursive solutions. Therefore, mastering recursion expands your arsenal of problem-solving strategies. It will nurture your logical thinking and make you a more efficient problem-solver, enhancing your attractiveness in the job market.

This course aims to help you understand the keys behind recursive thinking and problem-solving. You will go through the fundamental principles, pitfalls, and practical applications of recursion. You can expect several illustrated code examples to help you get a grip on recursion from a programmer's perspective.

So, get ready to step into the world of recursion, where all problems are smaller versions of themselves. Join us on this journey to grok the art of recursion!

Recursion Preliminaries

Let's start our journey to understand recursion by appreciating its roots and uses across many fields.

The term "recursion" is derived from Latin verb "recurrere", which means "to run back". Recurrere is a precise reflection of what recursion does. **A recursive solution runs back onto itself, solving a problem by breaking it down into sub-problems of the same nature.**

Recursion is a fundamental principle underlying numerous phenomena in nature and mathematics. Consider the beautiful yet mathematical structure of a fractal. Fractals, like the Mandelbrot set, are created by repeating a simple process over and over in an ongoing feedback loop. Similarly, in biology, we see recursion in the iterative process of DNA replication.

Fractal

As we proceed to the next sections, we'll translate this concept into computer programming, exploring how functions can call themselves to solve problems. We'll examine the architecture of a recursive function and identify some of the common pitfalls encountered in recursive implementations. So, let's run back and unravel the art of recursive problem-solving.

Structure of a recursive function

Understanding the structure of a recursive function is crucial for effective recursive problem-solving. Generally, if a function generates a call to itself in its body, we say it is a recursive function. However, a useful problem-solving recursive function should have well-defined definitions for the following constituents:

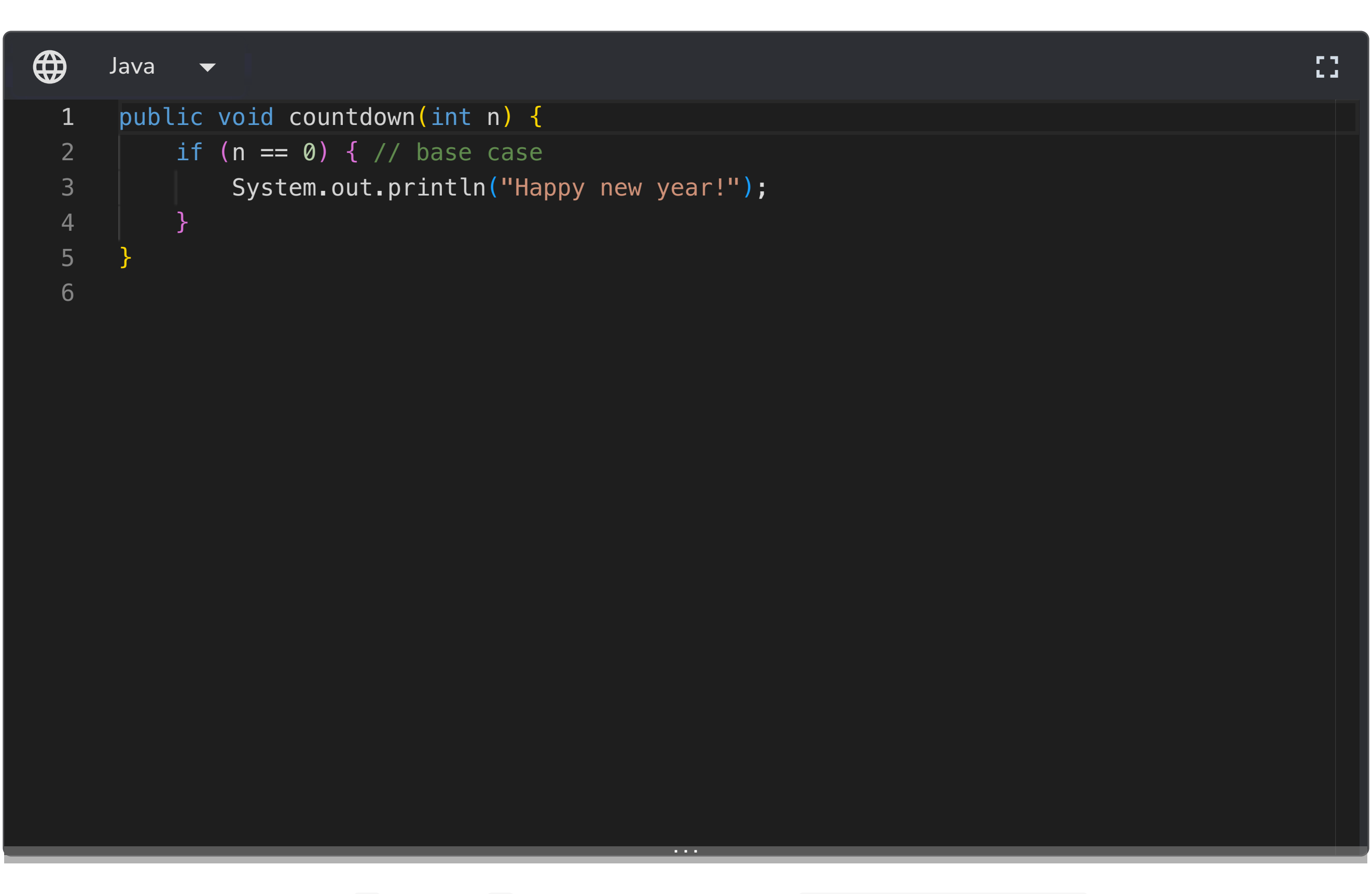
- Base case
- Recursive case
- Function prologue and epilogue

Let's understand these constituents with easy examples.

The base case

The base case is the condition under which the function stops making further recursive calls. You can consider it as the simplest instance of a problem that can be solved directly.

Let's consider a function that counts down from a given number `n`, and when it reaches the end, it prints the message 'Happy New Year!'.



```java
public void countdown(int n) {
    if (n == 0) { // base case
        System.out.println("Happy new year!");
    }
}
```

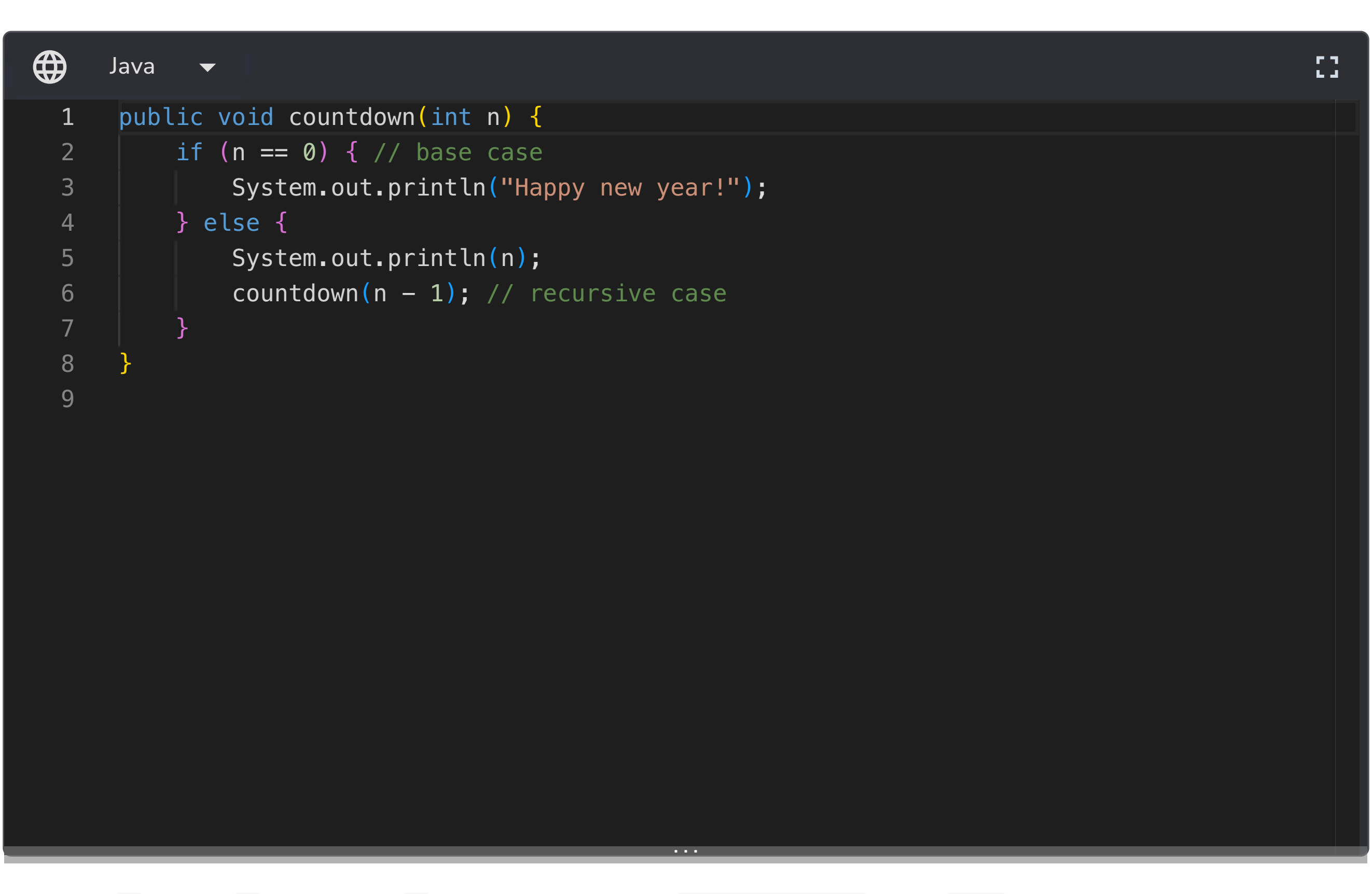
In the base case above, if `n` equals `0`, the function prints `"Happy new year!"`.

Important: Always begin by identifying the base case while designing a recursive function. It's the cornerstone that prevents infinite recursion.

The recursive case

It handles the more complex instances of the problem. It further breaks down the problem and calls the function with the reduced problem size.

Continuing with our countdown problem:



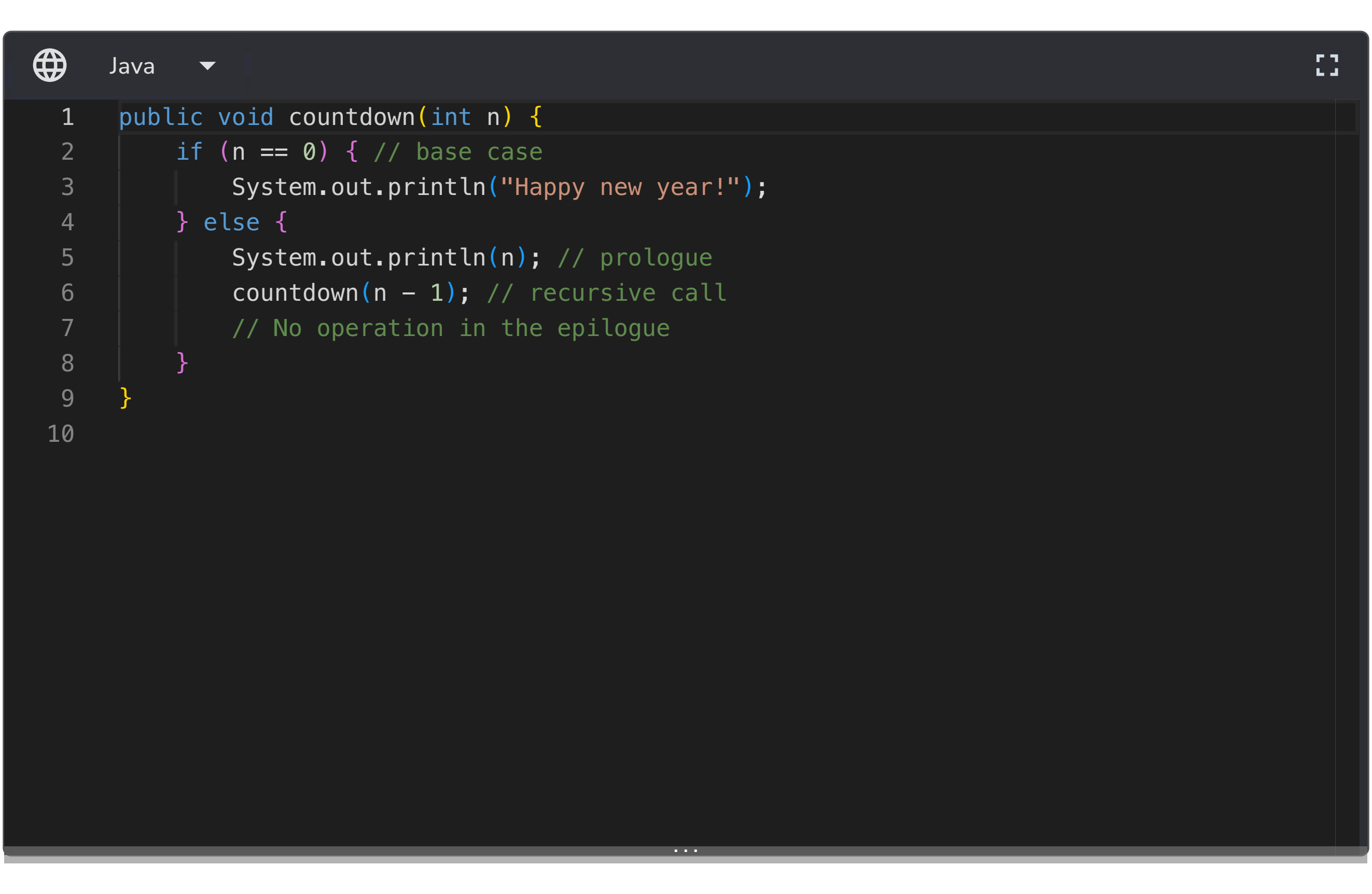
```java
public void countdown(int n) {
    if (n == 0) { // base case
        System.out.println("Happy new year!");
    } else {
        System.out.println(n);
        countdown(n - 1); // recursive case
    }
}
```

Here, if `n` is not `0`, we print `n` and then call the `countdown()` with `n-1`. Hence, all cases where we issue a new `countdown()` call are recursive cases.

Function prologue and epilogue

The prologue consists of the operations we perform before issuing a new recursive call, and the epilogue includes the operations we perform after the recursive call.

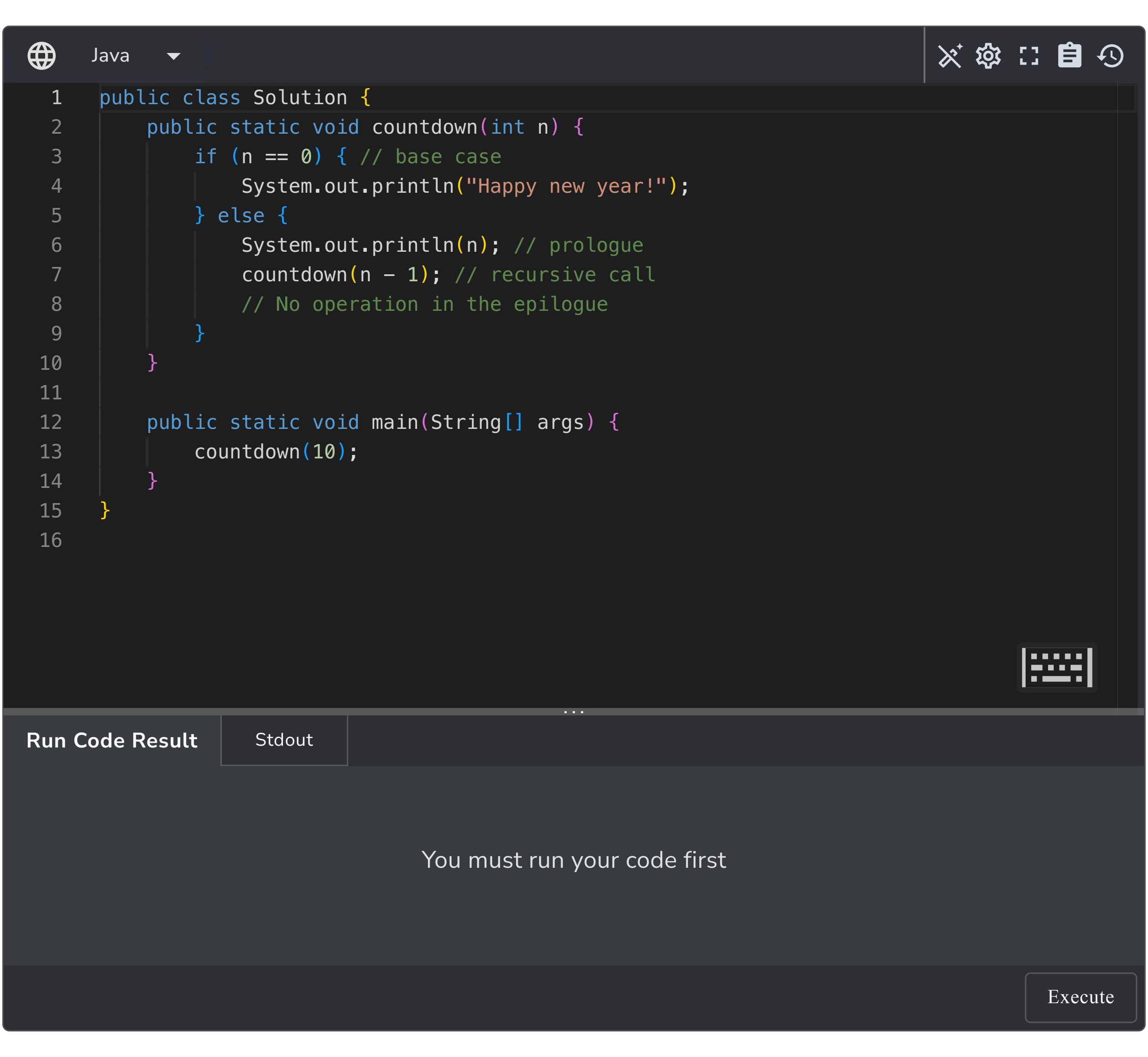
In our `countdown` example, printing the number `n` is part of the prologue, and there's no operation in the epilogue.



```java
public void countdown(int n) {
    if (n == 0) { // base case
        System.out.println("Happy new year!");
    } else {
        System.out.println(n); // prologue
        countdown(n - 1); // recursive call
        // No operation in the epilogue
    }
}
```

Be mindful of the operations in the prologue and epilogue. Their order can significantly affect the function's behavior.

Here is the complete working code example running countdown from 10:



```java
public class Solution {
    public static void countdown(int n) {
        if (n == 0) { // base case
            System.out.println("Happy new year!");
        } else {
            System.out.println(n); // prologue
            countdown(n - 1); // recursive call
            // No operation in the epilogue
        }
    }

    public static void main(String[] args) {
        countdown(10);
    }
}
```

You must run your code first

Challenge: Update the above `countdown()` function to have an epilogue instead of a prologue. Then try to figure out what changes in the final output.

Having understood the structure of recursive functions, it's important to note that incorrect implementation can lead to common errors. We'll navigate these potential pitfalls in the next section.

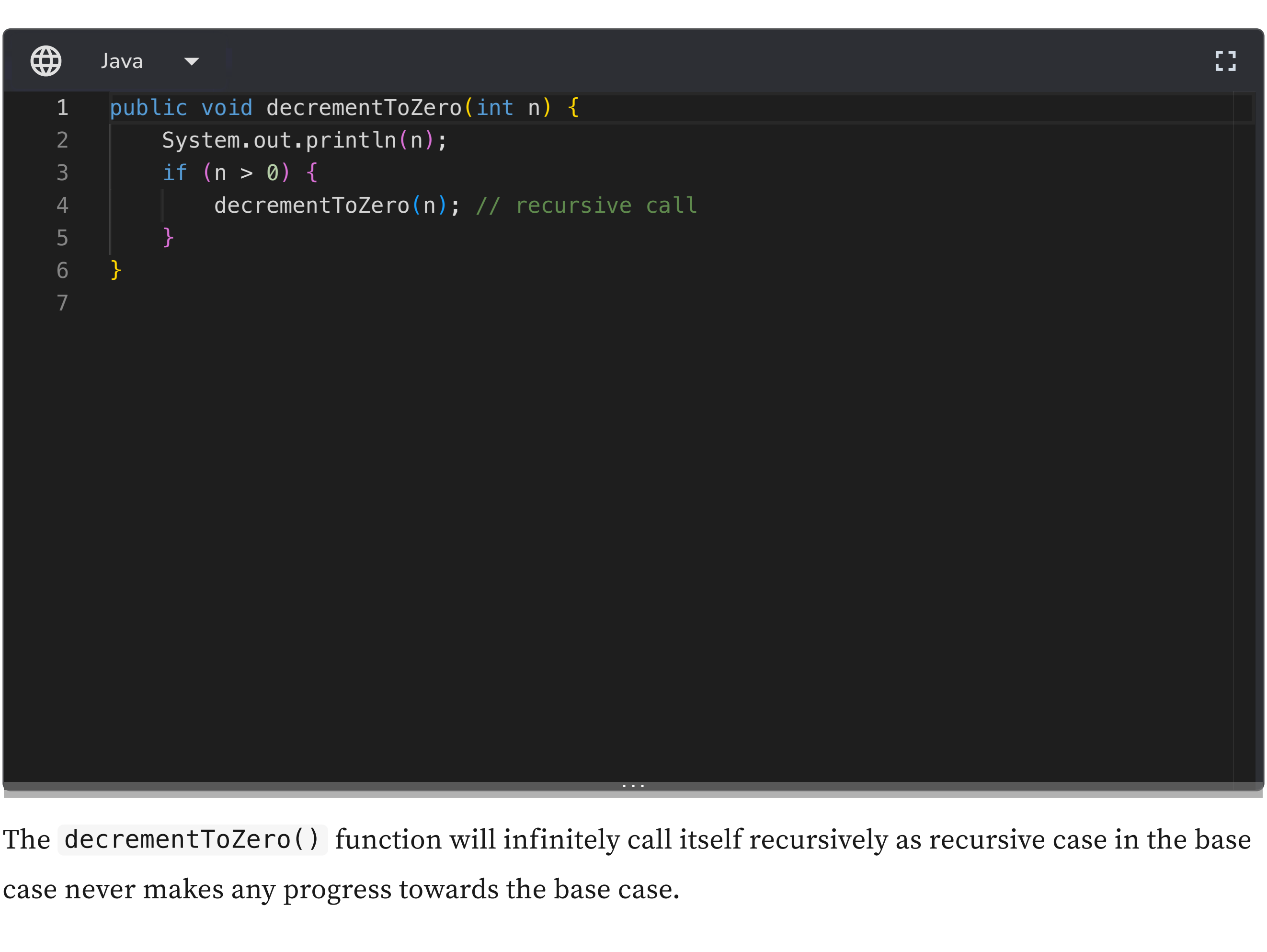
Common mistakes in recursive implementations

Even the most experienced programmers can occasionally stumble when it comes to recursion. It's a powerful tool, but one that requires a cautious approach. Let's examine a few common mistakes.

Infinite recursion

The first, and perhaps most dreaded, pitfall is infinite recursion, where a function calls itself endlessly. This happens when the base case isn't properly defined or reached.

Consider the following recursive function that should decrement a number `n` until it reaches `0`.



```java
public void decrementToZero(int n) {
    System.out.println(n);
    if (n > 0) {
        decrementToZero(n); // recursive call
    }
}
```

The `decrementToZero()` function will infinitely call itself recursively as recursive case in the base case never makes any progress towards the base case.

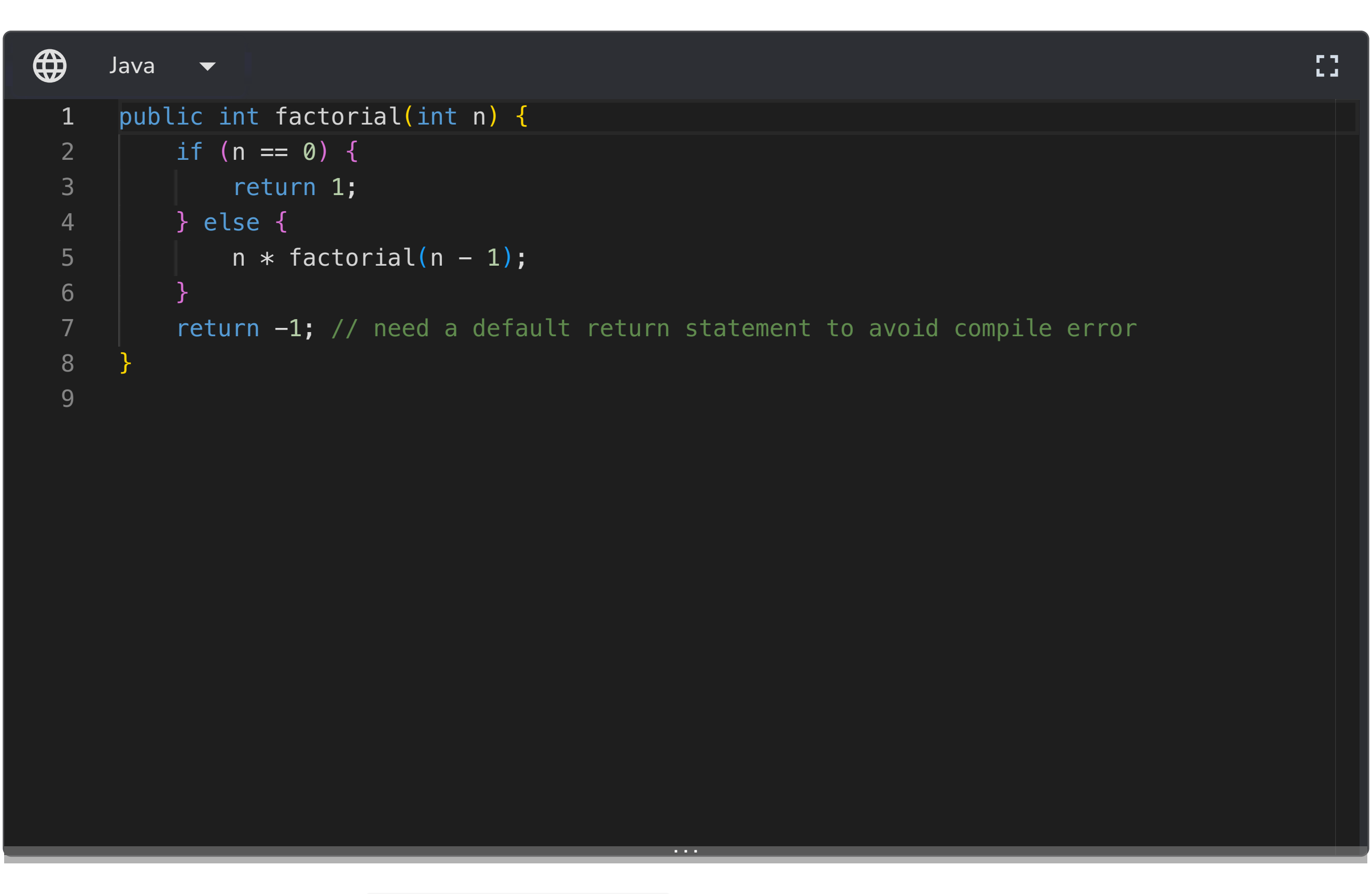
Tip: Always ensure your function makes progress towards its base case.

Challenge: Can you modify the `decrementToZero()` function to ensure it doesn't cause infinite recursion?

Answer: Just decrement `n` in the recursive call: `decrementToZero(n-1)`.

Ignoring the returned value

Another common oversight is ignoring the value returned by the recursive call, often leading to incorrect results. Here's an example:



```java
public int factorial(int n) {
    if (n == 0) {
        return 1;
    } else {
        n * factorial(n - 1);
    }
    return -1; // need a default return statement to avoid compile error
}
```

In the function above, the `n * factorial(n-1)` result isn't returned, leading to incorrect outputs.

Challenge: Can you fix the `factorial` function?

Answer: Add the return statement: `return n * factorial(n-1)`.

Excessive recursion

> In the context of recursion, a call stack is a mechanism that keeps track of function calls in a program. It's an important part of recursion because it helps keep track of where the program should return to after a function call is completed.

> When a function calls itself (which is what happens in recursion), each call to the function is added to the call stack. This includes the values of any parameters and variables. The program uses the call stack to keep track of these function calls.

> Then, when a base case is reached (which is the condition that stops the recursion), the program starts to "unwind" the stack. This means it starts to take function calls off the stack and finish them off one by one, using the variables and parameters values that were stored. This process continues until the stack is empty, which means all the function calls have been completed.*

Since each function call adds a stack frame over the call stack. Thereby, naively underestimating the recursion depth can lead to excessive load on the call stack. Hence, it can overload it - resulting in the stack-overflow exception.

As we delve deeper into this course, we'll learn more about these potential pitfalls and how to avoid them. After all, to master recursion is not just to understand its power but also to respect its challenges.

As we transition from understanding the common mistakes in recursive implementations, it's crucial to delve deeper into the functioning of recursive calls. You might be wondering, how do computers handle recursive calls? Or how can we visualize and test a recursive function?

In the upcoming section, we will address these questions. We'll explore the concept of the call stack and learn how to represent a recursive function using a recursion tree. So, let's demystify it.

Next →

Demystifying Recursion

☐ Mark as Completed